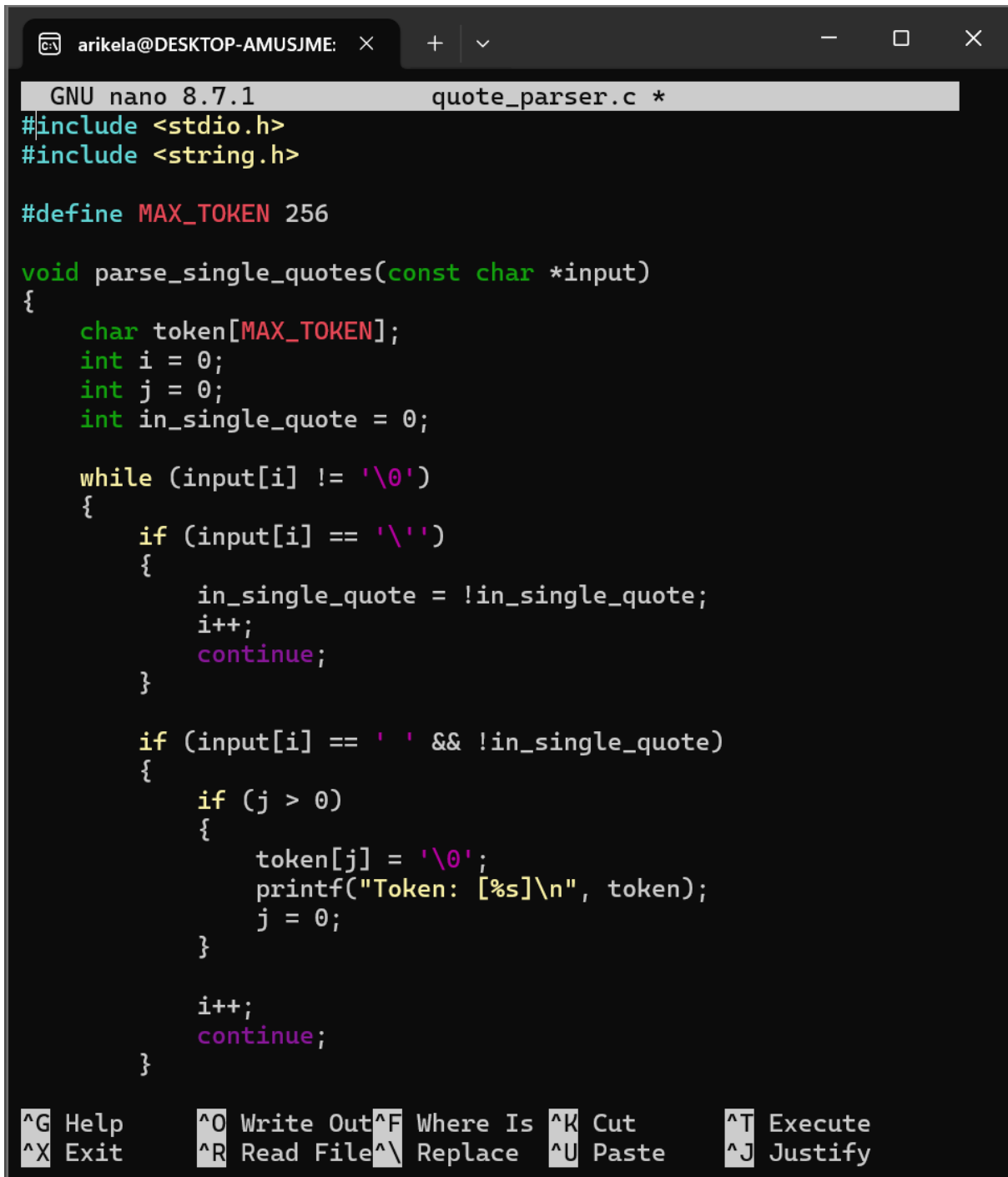


Skill Week-5

Task-1: TO Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases



```
GNU nano 8.7.1 quote_parser.c *
#include <stdio.h>
#include <string.h>

#define MAX_TOKEN 256

void parse_single_quotes(const char *input)
{
    char token[MAX_TOKEN];
    int i = 0;
    int j = 0;
    int in_single_quote = 0;

    while (input[i] != '\0')
    {
        if (input[i] == '\\')
        {
            in_single_quote = !in_single_quote;
            i++;
            continue;
        }

        if (input[i] == ' ' && !in_single_quote)
        {
            if (j > 0)
            {
                token[j] = '\0';
                printf("Token: [%s]\n", token);
                j = 0;
            }

            i++;
            continue;
        }
    }
}
```

Help **Write Out** **Where Is** **Cut** **Execute**
Exit **Read File** **Replace** **Paste** **Justify**

arikela@DESKTOP-AMUSJME: × + ▾

GNU nano 8.7.1 quote_parser.c *

```
        printf("Token: [%s]\n", token);
        j = 0;
    }

    i++;
    continue;
}

token[j++] = input[i++];
}

if (j > 0)
{
    token[j] = '\0';
    printf("Token: [%s]\n", token);
}
}

int main()
{
    char input[MAX_TOKEN];

    printf("Enter a command using single quotes:\n");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    printf("\nParsing result:\n");
    parse_single_quotes(input);

    return 0;
}
|
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^_\ Replace	^U Paste	^J Justify

```
arikel@DESKTOP-AMUSJME:~$ mkdir quote_parser
arikel@DESKTOP-AMUSJME:~$ cd quote_parser
arikel@DESKTOP-AMUSJME:~/quote_parser$ nano quote_parser.c
arikel@DESKTOP-AMUSJME:~/quote_parser$ nano quote_parser.c
arikel@DESKTOP-AMUSJME:~/quote_parser$ gcc quote_parser.c -o quote_parser
arikel@DESKTOP-AMUSJME:~/quote_parser$ ./quote_parser
Enter a command using single quotes:
echo 'hello world'

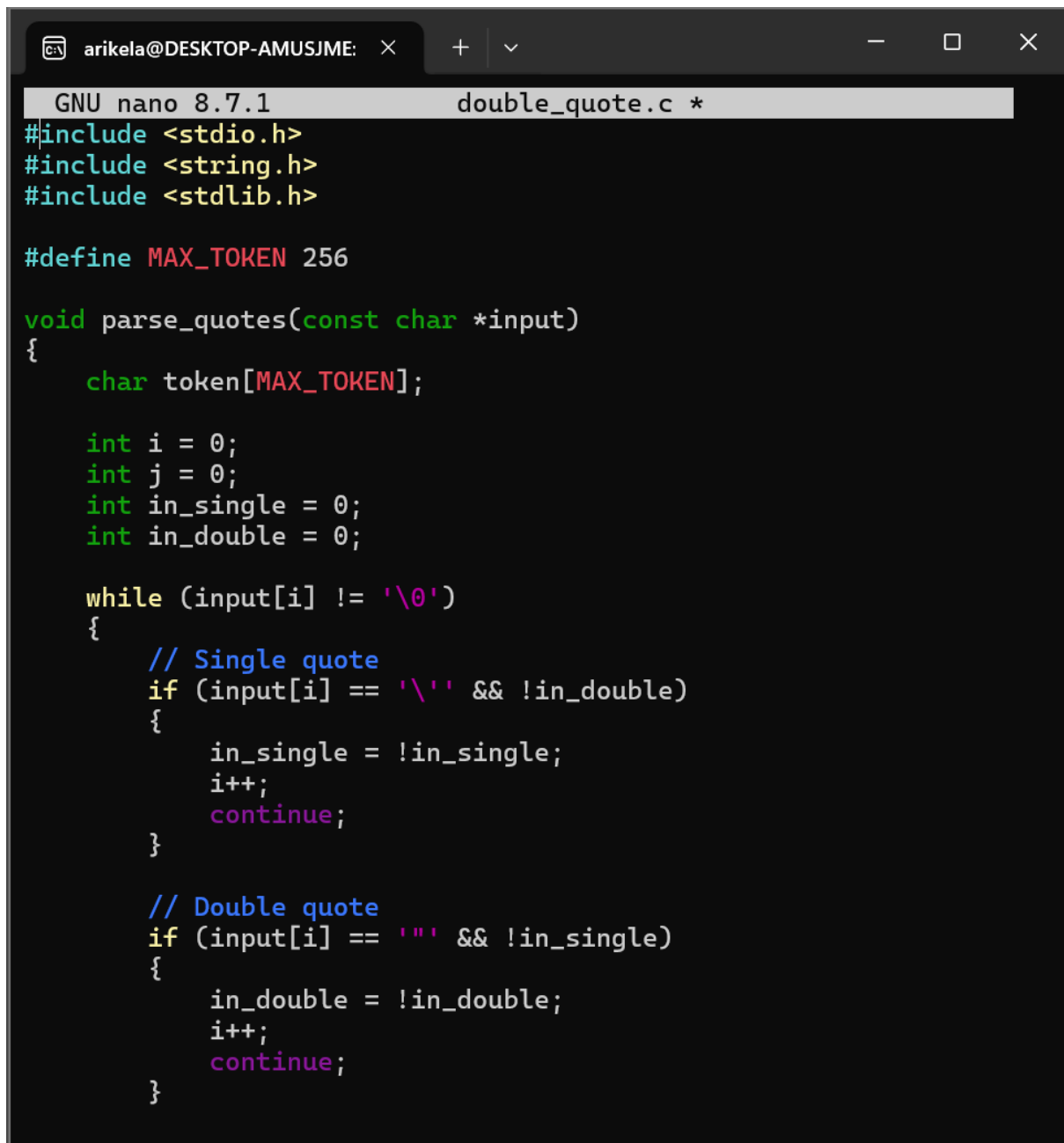
Parsing result:
Token: [echo]
Token: [hello world]
arikel@DESKTOP-AMUSJME:~/quote_parser$ ./quote_parser
Enter a command using single quotes:
echo '$HOME'

Parsing result:
Token: [echo]
Token: [$HOME]
arikel@DESKTOP-AMUSJME:~/quote_parser$ ./quote_parser
Enter a command using single quotes:
echo 'hello my friend'

Parsing result:
Token: [echo]
Token: [hello my friend]
arikel@DESKTOP-AMUSJME:~/quote_parser$ ./quote_parser
Enter a command using single quotes:
echo ''

Parsing result:
Token: [echo]
arikel@DESKTOP-AMUSJME:~/quote_parser$ |
```

Task-2: TO Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands



The image shows a terminal window with a dark background. At the top, there's a tab labeled 'arikela@DESKTOP-AMUSJME:'. Below the tab, the window title bar shows 'GNU nano 8.7.1' and the filename 'double_quote.c *'. The code is written in C and uses syntax highlighting: keywords are green, constants are red, strings are pink, and comments are blue. The code defines a macro for the maximum number of tokens and implements a function to parse quoted strings, handling both single and double quotes and toggling flags for each.

```
GNU nano 8.7.1 double_quote.c *
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#define MAX_TOKEN 256

void parse_quotes(const char *input)
{
    char token[MAX_TOKEN];

    int i = 0;
    int j = 0;
    int in_single = 0;
    int in_double = 0;

    while (input[i] != '\0')
    {
        // Single quote
        if (input[i] == '\'' && !in_double)
        {
            in_single = !in_single;
            i++;
            continue;
        }

        // Double quote
        if (input[i] == '"' && !in_single)
        {
            in_double = !in_double;
            i++;
            continue;
        }
    }
}
```

```
// Double quote
if (input[i] == '"' && !in_single)
{
    in_double = !in_double;
    i++;
    continue;
}

// Space outside quotes = token separator
if (input[i] == ' ' && !in_single && !in_double)
{
    if (j > 0)
    {
        token[j] = '\0';
        printf("Token: [%s]\n", token);
        j = 0;
    }

    i++;
    continue;
}

// Variable expansion inside double quotes
if (input[i] == '$' && !in_single)
{
    char variable[100];
    int k = 0;

    i++;

    while ((input[i] >= 'A' && input[i] <= 'Z') ||
           (input[i] >= 'a' && input[i] <= 'z') ||
```

^G Help**^X** Exit**^O** Write Out**^R** Read File**^F** Where Is**^_** Replace**^K** Cut**^U** Paste**^T** Execute**^J** Justify

GNU nano 8.7.1

double_quote.c *

```
// Variable expansion inside double quotes
if (input[i] == '$' && !in_single)
{
    char variable[100];
    int k = 0;

    i++;

    while ((input[i] >= 'A' && input[i] <= 'Z') ||
           (input[i] >= 'a' && input[i] <= 'z') ||
           (input[i] >= '0' && input[i] <= '9') ||
           input[i] == '_')
    {
        variable[k++] = input[i++];
    }

    variable[k] = '\0';

    char *value = getenv(variable);

    if (value != NULL)
    {
        for (int x = 0; value[x] != '\0'; x++)
            token[j++] = value[x];
    }

    continue;

    token[j++] = input[i++];
}

| if (j > 0)
```

^G Help

^O Write Out

^F Where Is

^K Cut

^T Execute

^X Exit

^R Read File

^_ Replace

^U Paste

^J Justify

```

    if (j > 0)
    {
        token[j] = '\0';
        printf("Token: [%s]\n", token);
    }
}

int main()
{
    char input[MAX_TOKEN];

    printf("Enter a command using quotes:\n");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    printf("\nParsing result:\n");

    parse_quotes(input);

    return 0;
}
|

```

[^]G Help
[^]X Exit

[^]O Write Out
[^]R Read File

[^]F Where Is
[^]\ Replace

[^]K Cut
[^]U Paste

[^]T Execute
[^]J Justify

```
arikel@DESKTOP-AMUSJME: x + v - □ ×  
arikel@DESKTOP-AMUSJME:~/quote_parser$ nano double_quote.c  
arikel@DESKTOP-AMUSJME:~/quote_parser$ gcc double_quote.c -o double_quote  
arikel@DESKTOP-AMUSJME:~/quote_parser$ ./double_quote  
Enter a command using quotes:  
echo "hello world"  
  
Parsing result:  
Token: [echo]  
Token: [hello world]  
arikel@DESKTOP-AMUSJME:~/quote_parser$ ./double_quote  
Enter a command using quotes:  
echo "$HOME"  
  
Parsing result:  
Token: [echo]  
Token: [/home/arikel]  
arikel@DESKTOP-AMUSJME:~/quote_parser$ ./double_quote  
Enter a command using quotes:  
echo "Hello Linux World"  
  
Parsing result:  
Token: [echo]  
Token: [Hello Linux World]  
arikel@DESKTOP-AMUSJME:~/quote_parser$ ./double_quote  
Enter a command using quotes:  
echo "hello 'linux' world"  
  
Parsing result:  
Token: [echo]  
Token: [hello 'linux' world]  
arikel@DESKTOP-AMUSJME:~/quote_parser$ ./double_quote  
Enter a command using quotes:  
echo ""  
  
Parsing result:  
Token: [echo]
```